Recap

1) Why inheritance ? adv of using this inheritance ?
So, one class can reuse the data members and methods of other class

2)When we can use extends kw
if the is a relationship is satisfied between two classes

3)why method overloading
if the functionality is same, instead of giving diff name to the methods
we can give the same name to the methods

4)method overloading and its cases
same method name
no of arguments
order of arguments
type of arguments

5)with method overloading, can we give diff return type
this is allowed.

Inheritance

When Dog class extends the Animal class, data members and methods of the Animal class is inherited to the Dog class.

How we can access those dm?
by creating the object for the sub class.

```
class Animal
{
   public String name;
   public int age;

   public void eat()
   {
```

```
        System.out.println(name + " is eating.");
    }


    public void display()
    {
        System.out.println("Name : " + name);
        System.out.println("Age  : " + age);
    }
}

class Dog extends Animal
{
          String breed;

}
```

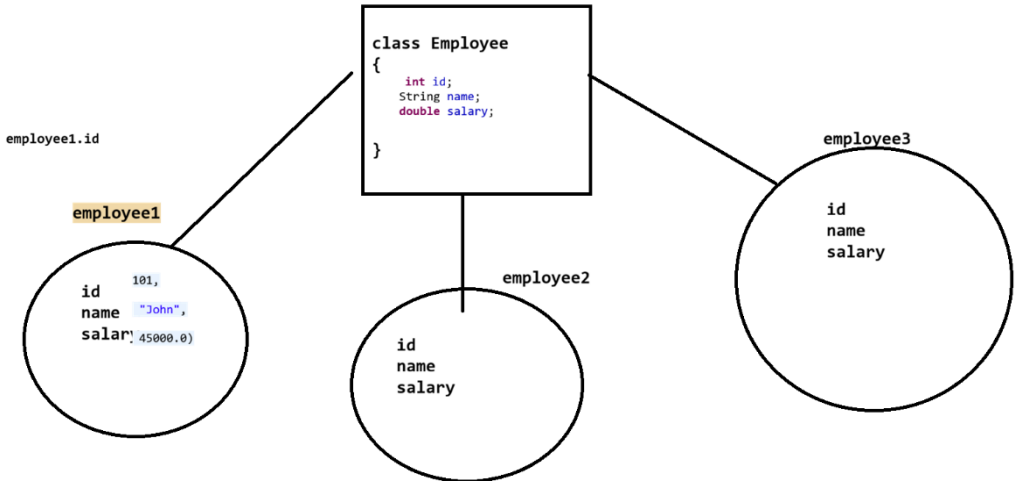
When Dog class extends the Animal class, Animal class just enables the permission to use its dm and methods.

In the Dog class, separate copy of Animal class dm and methods is maintained.

```
class Employee
{
 int id
 String name
 int age
}
```

when you create employee object, one instance will be created.

That object/instance will have its own copy of dm.

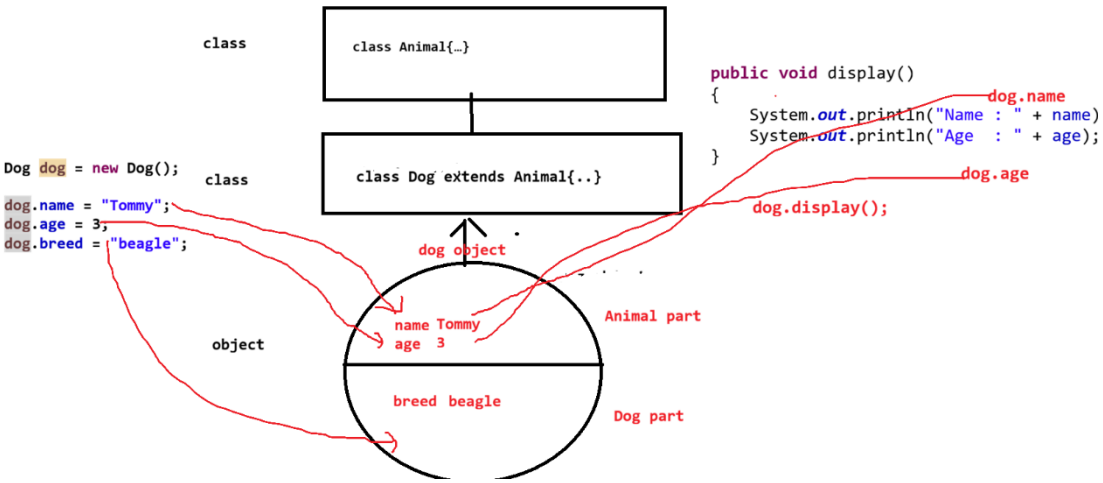


static members, object will not have its own copy of static data members.

static members are maintained in class level.

method
---
method also maintained in class level only(instance method and static method. But inside the static method, we can not access the instance variables.
)only one copy of method is maintained.

when you create the object for the Dog class, logically creates two parts
Animal part , Dog part.



Animal part will have its own copy of Animal class dm(instance variables)
Dog part will have its own copy of Dog class dm(instance variables)

method signature

method name and its type
return type is not included in the method signature.

method overriding?

1)inheritance - parent and child class relationship
2)method signature should be same including the return type - rule
3)after creating the object for the sub class, if you call the overridden method, then sub class overridden method will be invoked.
4)access specifier rule
3)Access modifier rule.

Sub class overridden method access modifier should not be more restrictive than super
class method access modifier
this points is only for reference purpose - this is not java points
40 : public: Unlimited access

30 : protected: Unlimited access with restrictions
20 : default: Limited access to a finite amount of other classes
10 : private: Extremely limited access: not accessible by anything .

1	public	public	allowed : 40 = 40
2	default	default	allowed : 20 = 20
3	protected	protected	allowed : 30 = 30
4	default	protected	allowed : 20 < 30
5	public	protected	Not allowed 40 > 30
6	protected	default	Not allowed. 30 > 20
8	public	private	Not allowed. 40 > 10
9	public	default	Not allowed. 40 > 20

Variable hiding:
If sub class uses the same dm name as super class, sub class dm hides the super class
dm even if the dm type is different.

super - dm
From the sub class, to access the hidden dm us super keyword.
from the sub class, to access the super class dm use super keyword.

super - method
from the sub class to access the super class method , use super k.w
super.methodname();

@Override annotation we can add it on the top of overridden method

Data hiding:
encapsulation
private dm

public getters and setters

when one class extends the other class, super class dm and methods are inherited to the sub class(instance var,instance method,static varible, static method) except private dm and private method.

Method overriding with instance method

instance method can be overridden
after creating the object for the sub class, and when you call the overridden method then sub class overridden method will be invoked.

static method is overridden or not?

static method can not be overridden.

```
class Animal
{
   static void eat()
   {
      System.out.println("Static Method from Animal");
   }

}

class Dog extends Animal
{
  here it seems like Dog class is overriding the Animal class eat method.
 Here Dog class considers this eat method as a separate new method.
```

To verify this method can be overridden or not add @Override annotation.

```
   static void eat()
   {
```

```
        System.out.println("Static Method from Dog");
    }

}
```

```
    Employee(int id, String name, double salary) {
            this(id, name);  - this should be the first stmt
        this.salary = salary;
```

difference between method overloading and method overriding

method overloading - happens in same class

method overriding - happens between parent and child class

Inheritance flow

```
Dog d1 = new Dog();
Animal Default Constructor
Dog Default Constructor
```

```
Dog d2 = new Dog("Tommy", 3, "Labrador");
Animal Default Constructor
Dog Parameterized Constructor
```